Software Requirements Specification

Next-Gen Equipment Management & Predictive Analytics System

IEEE 830 Standard — Version 1.0 DRAFT

Document Control

Field	Detail
Document Title	Software Requirements Specification — EMS v1.0
Document ID	EMS-SRS-2026-001
Project Title	Next-Gen Equipment Management & Predictive Analytics System
Organization	Heavy Industrial Construction Company — Oil & Gas / Refinery / Infrastructure Division
Prepared By	Surya Narayan C Shenoy
Reviewed By	Chandirasekharan, Santhosh Mohanan, Arun Dev
Document Version	1.0 DRAFT
Status	Pending Mentor Sign-Off
Created Date	01 - 07 - 2026
Last Revised	07 - 07 - 2026
Classification	Internal Confidential

Table of Contents

1. Purpose & Scope
    - 1.1 Purpose of This Document
    - 1.2 Scope of the System
    - 1.3 Definitions, Acronyms, and Abbreviations
    - 1.4 References
    - 1.5 Overview of the Remainder of This Document
2. Functional Requirements
    - 2.1 Overview of Functional Modules
    - 2.2 Module AR: Asset Registry
    - 2.3 Module DR: Driver & Operator Registry
    - 2.4 Module FM: Fuel Management
    - 2.5 Module MM: Maintenance Management
    - 2.6 Module ET: Equipment Transfer
    - 2.7 Module IM: Incident Management
    - 2.8 Module PS: Project & Site Management
    - 2.9 Module AD: Analytics Dashboard
    - 2.10 Module IL: Intelligence Layer
    - 2.11 Module SA: System Administration
3. Non-Functional Requirements
    - 3.1 Performance Requirements
    - 3.2 Scalability Requirements
    - 3.3 Security Requirements
    - 3.4 Availability and Reliability Requirements
    - 3.5 Usability Requirements
    - 3.6 Maintainability Requirements
    - 3.7 Regulatory Compliance Requirements
4. System Architecture
    - 4.1 Architectural Style
    - 4.2 Presentation Tier
    - 4.3 Application Tier
    - 4.4 Data Tier
    - 4.5 Machine Learning Service
    - 4.6 Scheduled Batch Jobs
    - 4.7 Authentication Architecture
    - 4.8 File Storage Architecture
5. Data Management
    - 5.1 Data Architecture Principles
    - 5.2 Database Normalization Standard
    - 5.3 Primary Key Strategy
    - 5.4 Indexing Strategy
    - 5.5 Data Validation Rules
    - 5.6 Data Retention Policy
    - 5.7 Seed Data Strategy
6. User Documentation Requirements
    - 6.1 In-Application Help
    - 6.2 User Role Guides

- 6.3 System Administrator Guide
- 6.4 API Documentation
- 6.5 Data Dictionary

7. Testing Requirements
   - 7.1 Testing Strategy Overview
   - 7.2 Unit Testing Requirements
   - 7.3 Integration Testing Requirements
   - 7.4 System Testing Requirements
   - 7.5 User Acceptance Testing Requirements
8. Acceptance Criteria
   - 8.1 Database and Schema Criteria
   - 8.2 Application Functionality Criteria
   - 8.3 Dashboard Criteria
   - 8.4 Machine Learning Criteria
   - 8.5 Code Quality Criteria
9. Project Timeline (WBS)
   - 9.1 Overview
   - 9.2 Summary Timeline

Section 1 — Purpose & Scope

1.1 Purpose of This Document

This Software Requirements Specification defines the complete, unambiguous, and verifiable set of requirements for the Next-Gen Equipment Management and Predictive Analytics System, hereafter referred to as EMS. This document is prepared in accordance with the IEEE 830-1998 standard for software requirements specifications.

The purpose of this document is fourfold. First, it establishes a shared understanding between the development team (the intern), the mentoring engineers, and the end-user stakeholder groups regarding what the system will do, how it will behave, and under what constraints it will operate. Second, it serves as the contractual baseline against which all delivered software will be evaluated during User Acceptance Testing. Third, it provides the architectural foundation from which all subsequent design documents, database schemas, API specifications, and machine learning model briefs will be derived. Fourth, it acts as the primary reference for the final internship evaluation, demonstrating that the system was built from a rigorous, documented engineering process rather than ad-hoc development.

This document is intended to be read by the following audiences: the intern developer who will implement the system, the three mentors who will review and approve the design, the simulated stakeholder groups (Fleet Manager, Site Engineers, Workshop Mechanics, HSE Officers, Finance, Executive Management) whose operational needs are encoded here, and any future developer who picks up this codebase for enhancement or maintenance.

1.2 Scope of the System

The Equipment Management and Predictive Analytics System is a web-based, full-stack software platform designed for a large-scale industrial construction company operating in the Oil and Gas, Refinery, and Infrastructure sectors across the Gulf region. The company manages a diverse fleet of over 50 asset types spanning executive vehicles, personnel transport buses, heavy earthmoving machinery, lifting equipment, and static auxiliary power assets deployed across multiple concurrent construction project sites.

The EMS will serve as the single source of truth for the entire asset lifecycle of every piece of equipment and vehicle in the company's fleet. This lifecycle begins at asset acquisition and registration, moves through operational deployment across project sites, captures all fuel consumption, daily utilization, and operator assignments at the shift level, tracks every maintenance event whether scheduled preventive or unplanned corrective, records all equipment transfers between sites with full chain-of-custody documentation, logs every accident and safety incident tied to specific assets and operators, and concludes with asset decommissioning or write-off.

Beyond record-keeping, the EMS will embed intelligence into fleet operations through two mechanisms. The first is an executive analytics dashboard delivering real-time and historical Key Performance Indicators covering fleet utilization rate, mean time between failures, mean time to repair, total cost of ownership, and fuel consumption trends. The second is a machine learning intelligence layer that generates predictive maintenance risk scores per asset, forecasts site-level fuel consumption requirements, profiles driver behavior risk, and triggers automated replacement consideration alerts when an asset's repair cost trajectory becomes economically unviable.

The system will enforce role-based access control aligned to five primary operational roles, ensuring that each user group sees and can modify only the data that is relevant and authorized for their function. It will comply with Gulf-region regulatory requirements for vehicle registration, driver licensing, equipment certification, and incident reporting documentation.

Explicitly within scope: asset master registry with full Gulf regulatory and financial data, driver and operator registry with license and behavioral tracking, fuel management with anomaly detection, preventive and corrective maintenance management, equipment transfer workflow with multi-level approval, HSE incident reporting, site fuel tank inventory and reconciliation, tyre and battery component tracking, lifting equipment certification management, executive KPI dashboard, predictive maintenance classification model, fuel consumption regression forecasting model, driver behavior scoring model, automated replacement alert engine, document attachment management, and a unified system alert and notification center.

Explicitly outside scope for this twelve-week prototype: live GPS telematics integration, full ERP or HR system integration, mobile native application, full procurement and inventory management module, and payroll or HR management.

Context Diagram Level 0

Figure 1.1: Global Functional Boundary and External Entities

1.3 Definitions, Acronyms, and Abbreviations

Asset — Any piece of equipment, vehicle, or machinery owned, leased, or hired by the company and tracked within the EMS.

EMS — Equipment Management and Predictive Analytics System.

RBAC — Role-Based Access Control. A security model where system permissions are assigned to roles rather than individuals, and users are assigned to roles.

CRUD — Create, Read, Update, Delete. The four fundamental operations on any data record.

KPI — Key Performance Indicator. A measurable value that demonstrates how effectively operational objectives are being achieved.

MTBF — Mean Time Between Failures. The average operational time between one unplanned breakdown and the next for a given asset or asset class.

MTTR — Mean Time To Repair. The average elapsed time from breakdown report to the asset being returned to operational service.

TCO — Total Cost of Ownership. The sum of all costs associated with an asset across its lifetime: purchase price, fuel costs, maintenance costs, insurance premiums, and operator costs.

Fleet Utilization Rate — The ratio of actual productive operating hours to total available hours, expressed as a percentage.

Mulkiya / Istimara — The Arabic term for the vehicle registration certificate issued by the Department of Transportation in GCC countries.

DOT — Department of Transportation.

GCC — Gulf Cooperation Council. Bahrain, Kuwait, Oman, Qatar, Saudi Arabia, and the UAE.

OEM — Original Equipment Manufacturer (Caterpillar, Komatsu, Liebherr, etc.).

Job Card — The physical and digital record of a maintenance activity.

Chain of Custody — The documented, chronological record of every transfer of an asset from one location or responsible party to another.

Predictive Maintenance — A data-driven maintenance strategy where machine learning models analyze historical operational data to forecast the probability of equipment failure before it occurs.

HSE — Health, Safety, and Environment.

ERD — Entity-Relationship Diagram.

API — Application Programming Interface.

JWT — JSON Web Token. A compact, self-contained token used for secure authentication between the client and the API server.

UAT — User Acceptance Testing.

F1-Score — A machine learning evaluation metric that combines precision and recall into a single number.

R^2 — Coefficient of Determination. A statistical metric indicating how well a regression model's predictions match actual observed values.

Soft Delete — A deletion pattern where a record is marked as archived or inactive rather than physically removed from the database.

LOLER — Lifting Operations and Lifting Equipment Regulations.

1.4 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- Project Assignment Document — Next-Gen Equipment Management and Predictive Analytics System, Internship Brief provided by Chandirasekharan, Santhosh Mohanan, and Arun Dev.
- EMS Requirements Engineering Blueprint v1.0 — Internal document produced during Week 1 requirements gathering.
- EMS Deliverable 1 — Entity List and All Relationships v1.0 — Internal document defining all 45 database entities and 71 inter-entity relationships.
- GCC Department of Transportation Vehicle Registration Regulations — Applicable regulations for Qatar, UAE, Saudi Arabia, Oman, Kuwait, and Bahrain.
- LOLER 1998 and equivalent GCC lifting equipment certification standards.

1.5 Overview of the Remainder of This Document

Section 2 defines all functional requirements organized by module. Section 3 defines all non-functional requirements covering performance, security, scalability, and compliance. Section 4 describes the system architecture. Section 5 covers data management policies. Section 6 defines user documentation requirements. Section 7 specifies testing requirements. Section 8 defines acceptance criteria. Section 9 presents the project timeline as a Work Breakdown Structure.

Section 2 — Functional Requirements

2.1 Overview of Functional Modules

The system's broad operational capabilities are mapped to isolated functional components. The global interaction boundaries between system roles, external handlers, and software services can be reviewed via our high-level functional model:

Click Here to View Full Use Case Diagram in High Definition

Figure 2.1: Global Functional Boundary and User Actor Use Cases

The EMS functional requirements are organized into ten modules. Each requirement is assigned a unique identifier in the format FR-[MODULE CODE]-[NUMBER]. Priority levels: **Critical** (system cannot function without it), **High** (significant operational impact if absent), **Medium** (improves usability or completeness), **Low** (enhancement that does not affect core operation).

The ten modules are: Asset Registry (AR), Driver & Operator Registry (DR), Fuel Management (FM), Maintenance Management (MM), Equipment Transfer (ET), Incident Management (IM), Project & Site Management (PS), Analytics Dashboard (AD), Intelligence Layer (IL), and System Administration (SA).

2.2 Module AR: Asset Registry

FR-AR-001 — Asset Registration `Critical`

The system shall allow authorized users with Fleet Manager or System Admin role to create a new asset record. The creation form shall capture all mandatory fields: internal asset number (auto-generated EQ-YYYY-NNNN), asset category, asset sub-type, make, model, year of manufacture, and ownership type. Duplicate asset numbers are prevented.

FR-AR-002 — Engine Specification Recording `Critical`

Upon creation, the system shall simultaneously create an associated engine specification record: engine make/model, serial number, chassis/frame serial number, fuel type, rated horsepower, torque, displacement, engine configuration, emission standard, transmission type, drive configuration, cooling system type, and OEM-specified service intervals.

FR-AR-003 — Gulf Registration Recording `Critical`

For Executive/Light Commercial and Personnel Transport categories, a Gulf Registration record is mandatory: plate number, country of registration, registration certificate number, registration expiry date, and traffic file number. Plate number format is validated against the selected country.

FR-AR-004 — Purchase Record Entry `High`

Finance and Fleet Manager roles can enter a purchase record: PO number, vendor reference, purchase date, purchase price, depreciation method, useful life (years), residual value, current book value (auto-calculated), capitalization date, and funding source. If lease/finance, additional fields: lessor name, monthly payment, lease end date.

FR-AR-005 — Insurance Policy Recording `Critical`

Insurance policy records per asset: policy number, provider, type, coverage start/end dates, premium, insured value, and broker contact. One policy may cover multiple assets. Coverage date overlaps for the same asset and policy type are prevented.

FR-AR-006 — Asset Status Management `Critical`

Valid statuses: Active, Under Maintenance, Idle, In Transit, Decommissioned, Written Off. Transition rules enforced: cannot move to In Transit while Under Maintenance; cannot log fuel/utilization for Decommissioned/Written Off assets; only Fleet Manager and System Admin may set Decommissioned or Written Off.

FR-AR-007 — Asset Search and Filtering `Critical`

Searchable, filterable asset list view with current status, site, category, make, model, and alert count. Filters by category, sub-type, status, site, and ownership type. Partial string search on asset number, make, model, and serial number.

FR-AR-008 — Asset Detail View `Critical`

Comprehensive single-asset page displaying all linked entity data: master record, engine spec, Gulf registration, purchase record, insurance policies, site/operator assignment, maintenance history, fuel summary, incident history, active alerts, predictive maintenance score, and document attachments. All sections accessible via tabs or collapsible panels.

FR-AR-009 — Document Attachment `High`

Authorized users can upload documents to any asset record. Supported types: Registration Certificate, Insurance Policy, Purchase Invoice, OEM Service Manual, Inspection Report, Certification Document, Photograph. Formats: PDF, JPG, PNG, DOCX. Max 10 MB per file. Attachments display uploader name, date, and document type.

FR-AR-010 — Asset Write-Off Process `High`

Structured write-off workflow: write-off date, reason, insurance claim number (if applicable), salvage value, and finance department notification confirmation. On completion: status set to Written Off, active assignments closed, pending transfers auto-rejected, asset excluded from active views but retained with full history.

FR-AR-011 — Tyre Record Management `Medium`

For wheeled assets: track individual tyres by serial number, wheel position, installation date, meter reading, tread depth, current status (Fitted/Removed/Scrapped), removal date, removal meter reading, and reason. Visual wheel position diagram per asset.

FR-AR-012 — Battery Record Management `Medium`

Track batteries per asset: serial number, brand, installation date, removal date, and replacement reason. History retained indefinitely.

FR-AR-013 — Equipment Certification Management `Critical`

Lifting and Heavy Shifting Equipment must have at least one valid certification before being set to Active. Certification record: certificate number, issuing authority, test date, expiry date, safe working load, and inspection vendor. Alerts generated at 60, 30, and 7 days before expiry.

FR-AR-014 — Expiry Alert Generation `Critical`

Automatic SystemAlert records generated at the following intervals before expiry: Gulf Registration at 90/60/30 days; Insurance Policy at 90/60/30 days; Equipment Certification at 60/30/7 days; Driver License at 60/30/7 days; Driver Medical Fitness at 60/30/7 days. All alerts visible on Fleet Manager dashboard with in-app notification.

A. Behavioral Process Streams & Data Flows

Click to Open High-Resolution Asset Registration DFD ↗

B. Asset Operational States

The structural status flags (`Active` , `Idle` , `Under Maintenance` , `Decommissioned`) and transition boundaries are tracked via the operational engine:

- **State Transitions:** View High-Resolution State Machine 1 — Asset Lifecycle Diagram ↗

2.3 Module DR: Driver & Operator Registry

FR-DR-001 — Driver Registration `Critical`

Mandatory fields: employee ID, full name, nationality, date of birth, job title, license number, license category, issuing authority, license expiry, medical fitness certificate number, and medical fitness expiry. Optional: years of experience, previous employer, emergency contact.

FR-DR-002 — License Category Validation `Critical`

Lookup table enforcing eligible asset sub-types per license: Light Vehicle — Executive/Light Commercial only; Heavy Vehicle — Personnel Transport and Heavy Earthmoving; Crane Operator Certificate — all crane sub-types; Forklift Operator Certificate — all forklifts; Aerial Work Platform Certificate — Boom Lifts and Telescopic Work Platforms. Validated at driver-to-asset assignment.

FR-DR-003 — Training Record Entry `High`

Training events per driver: type, provider, date, certificate number, and expiry (if applicable). Completed training updates driver's eligible assignment scope.

FR-DR-004 — Driver Behavior Score Display `High`

Composite behavior score displayed on driver profile broken down into: incident score (40%), fuel overconsumption score (30%), breakdown attribution score (20%), and compliance score (10%). Risk category displayed with color coding: green (Low), amber (Medium), red (High). 12-month trend chart shown.

FR-DR-005 — High-Risk Driver Assignment Block `High`

Warns or blocks (configurable) assigning a High Risk driver to: Executive/Light Commercial VIP vehicles, Crawler Cranes, and All-Terrain Cranes. Fleet Manager override requires a logged reason.

FR-DR-006 — Expired License Assignment Block `Critical`

Prevents assignment of an expired-license driver to any asset. If license expires mid-assignment, a Critical SystemAlert is generated. The asset remains operable (not blocked) but the alert must be

acknowledged before the next utilization log entry.

FR-DR-007 — Driver Search and List View `High`

Driver list showing: name, employee ID, license category, license expiry, current asset assignment, current site, and risk category badge. Filters: license category, risk category, site, and license expiry within N configurable days.

2.4 Module FM: Fuel Management

FR-FM-001 — Fuel Log Entry `Critical`

Site Engineer and Workshop Mechanic roles create fuel log entries: asset ID, operator on duty, project site, date/time, fuel type, quantity (liters), meter reading, fuel source, voucher number, unit price, and total cost. Fuel type must match engine specification.

FR-FM-002 — Fuel Tank Capacity Validation `Critical`

Quantity entered is validated against engine specification tank capacity. Entry is blocked if quantity exceeds capacity, displaying the maximum tank capacity.

FR-FM-003 — Fuel Efficiency Auto-Calculation `High`

Fuel efficiency auto-calculated from (current meter reading − previous meter reading) ÷ quantity. Expressed in km/L for vehicles and L/operating hour for equipment. Stored on the fuel log record.

FR-FM-004 — Fuel Anomaly Detection `High`

Rolling 30-day average efficiency computed after each entry. If deviation exceeds configurable threshold (default 20%), a FuelAnomaly record is created and a SystemAlert sent to Fleet Manager with expected range, actual value, and percentage deviation.

FR-FM-005 — Site Fuel Tank Management `High`

Fleet Manager and Site Engineer record bulk fuel deliveries: tank identifier, fuel type, delivery date, quantity, supplier, delivery note number, unit price, and total cost. Running balance maintained (deliveries minus dispensed).

FR-FM-006 — Fuel Reconciliation Report `High`

Per-site, per-period report comparing total delivered vs. total dispensed. Unaccounted fuel shown in liters and monetary value. Variance above configurable threshold generates a SystemAlert.

FR-FM-007 — Fuel Cost Dashboard `High`

Fuel cost trend lines per site per week/month. Shows fuel cost by asset class, top five highest fuel-consuming assets, and year-on-year comparison where historical data is available.

A. Fuel Telemetry Streams & Process Mapping

- **Process Streams:** View DFD 2 — Fuel Management Diagram ↗

B. Runtime Fuel Log Processing & Exception Rules

The validation execution loops for incoming log receipts, fuel capacity limits, and anomaly detection flags run in this exact sequence:

Click to Open High-Resolution Fuel Log Validation Sequence ↗

2.5 Module MM: Maintenance Management

FR-MM-001 — Preventive Maintenance Schedule Generation `Critical`

Schedule records auto-generated per asset from OEM service intervals: engine oil change, oil filter, air filter, fuel filter, hydraulic oil change, greasing schedule, full OEM inspection. Next due date calculated from last service date plus OEM interval.

FR-MM-002 — Overdue Maintenance Flagging `Critical`

Asset flagged overdue when current meter reading exceeds next due by more than a configurable threshold (default: warning at 5%, critical at 10%). Color-coded urgency on maintenance dashboard. SystemAlert generated to Fleet Manager and Workshop Supervisor.

FR-MM-003 — Job Card Creation `Critical`

Workshop Mechanic and Supervisor create job cards: asset ID, type (Preventive/Corrective), service type or fault description, date opened, current meter reading, assigned technicians, and workshop location. Corrective cards must link to a BreakdownLog. External cards require a vendor selection.

FR-MM-004 — Parts Consumption Recording `Critical`

Parts line items on job cards: part number, description, quantity, unit cost, and total cost. No limit on line items. Total parts cost auto-summed.

FR-MM-005 — Labor Hours Recording `High`

Labor entries per job card: technician employee ID, date, hours worked, labor rate per hour, and total cost. Multiple technicians may log against one job card. Total labor cost auto-summed.

FR-MM-006 — Job Card Status Workflow `Critical`

Statuses in order: Open → In Progress → Parts Pending → Completed Pending Approval → Closed. Only Workshop Supervisor may Close a card (requires sign-off remark). Closed cards are read-only. Parts Pending requires reason and expected delivery date.

FR-MM-007 — Next Service Recalculation on Job Card Close `Critical`

On closure, next due date is recalculated: (meter reading at close) + (OEM interval). If corrective work includes a component with its own schedule (e.g., oil change during breakdown repair), the preventive schedule is updated accordingly.

FR-MM-008 — Repair Warranty Recording `Medium`

For external vendor job cards: warranty description, start date, end date, and vendor warranty contact. If a new breakdown occurs within the warranty period for the same fault category, a SystemAlert flags it as a potential warranty claim.

FR-MM-009 — Maintenance Cost Summary Per Asset `High`

Running total of all maintenance costs per asset: preventive costs, corrective costs, parts, labor, and external vendor costs. Feeds directly into TCO calculation and Replacement Alert engine.

FR-MM-010 — Breakdown Log Creation `Critical`

Site Engineer and Workshop Mechanic create breakdown logs: asset ID, date/time, location, operator on duty, symptom description, and fault category (Engine, Hydraulics, Electrical, Structural, Tyres, Brakes, Transmission, Cooling System, Other). Response Time = time from breakdown log creation to job card creation (feeds MTTR).

FR-MM-011 — Maintenance Transfer Block `Critical`

Transfer dispatch is blocked while asset status is Under Maintenance. Block is overridable by Fleet Manager only with a mandatory logged reason.

A. Maintenance Scheduling Data Flows

- **Data Process Streams:** View DFD 3 — Maintenance Management Diagram ↗

B. Runtime Execution & Job Ticket Management

The runtime workflow for dispatching engineers, validating spare parts stock availability, and updating equipment downtime metrics follow this execution:

Click to Open Maintenance Ticket Sequence Diagram ↗

C. Technical Job Card Status Lifecycles

The automated state tracking machine shifts work orders through the system pipeline (`Opened` → `In Progress` → `Closed`):

- **Operational Lifecycle States:** View State Machine 2 — Job Card Lifecycle Diagram ↗

2.6 Module ET: Equipment Transfer

FR-ET-001 — Transfer Request Submission `Critical`

Site Engineer and Project Manager submit transfer requests: asset ID, source location, destination location, and reason (Project Mobilization, Demobilization, Reallocation, Breakdown Recovery, Return to Yard, or Other with free text). System prevents submission if a pending or approved transfer already exists for the asset.

FR-ET-002 — Transfer Approval Workflow `Critical`

Pending requests appear in Fleet Manager approval queue. Approve or Reject with mandatory remark. Approved → SystemAlert to requester. Rejected → remark displayed to requester.

FR-ET-003 — Gulf Regulatory Compliance Check on Transfer `Critical`

Before approval, system auto-verifies: Gulf Registration valid, Insurance Policy active, and (if driver assigned) driver license valid. Failed checks shown with expiry dates. Approval blocked until issue is resolved or Fleet Manager overrides with a logged reason.

FR-ET-004 — Gate Pass Reference Recording `High`

When moving to Dispatched status: gate pass number, departure date/time, assigned driver (if company driver), and transporter details if third-party (company name, vehicle plate, driver name, waybill number).

FR-ET-005 — Pre-Departure Inspection `High`

Required before dispatch confirmation: tyre condition, lights operational, fluid levels, body damage noted, fuel level at departure, meter reading, and overall condition rating. Minimum one photograph attachment is mandatory.

FR-ET-006 — Arrival Confirmation and Post-Arrival Inspection `Critical`

Receiving Site Engineer confirms arrival date/time and completes post-arrival inspection (same format). Condition worse than departure auto-generates a SystemAlert for potential transit damage. Asset site assignment is updated and transfer status set to Completed.

FR-ET-007 — In-Transit Breakdown Handling `High`

Breakdown log created for an In Transit asset links to the active transfer request. Transfer status set to In Transit — Breakdown. Fleet Manager alerted immediately. Asset status set to Under Maintenance. Transfer remains open and resumable.

FR-ET-008 — Transfer History View `High`

Complete chain of custody log per asset: transfer ID, source, destination, approved by, departure/arrival dates, gate pass number, duration in transit, and inspection ratings. Viewable on asset detail page and exportable as PDF.

FR-ET-009 — Cross-Border Transfer Documentation `Medium`

Additional informational fields for cross-GCC transfers: Carnet de Passage number, export certificate reference, and customs clearance reference. Informational only in this phase; not system-enforced.

A. Relocation Logistics and Incident Streams

The asset dispatch flows, route constraints, cross-border checklist clearings, and telemetry collision metrics map to these paths:

- **Transfer Data Flows:** View DFD 4 — Equipment Transfer Diagram ↗
- **Asset Relocation Lifecycles:** View State Machine 3 — Transfer Lifecycle Diagram ↗

2.7 Module IM: Incident Management

FR-IM-001 — Incident Report Creation `Critical`

HSE Officer, Site Engineer, and Fleet Manager create incident reports: asset ID, driver/operator ID, date/time, location, and incident type (Minor Accident, Major Accident, Near Miss, Fire, Equipment Tip-Over, Falling Object, Third-Party Property Damage, or Personal Injury).

FR-IM-002 — Third-Party Incident Data `High`

If a third party is involved: third-party vehicle plate, company name, driver name, and insurance details. Mandatory for Minor Accident, Major Accident, or Third-Party Property Damage types.

FR-IM-003 — Police Report and Insurance Claim Linkage `High`

Fields for police report number (Muroor/Traffic Department reference) and insurance claim number. Warning displayed if Major Accident or Third-Party Property Damage is closed without a police report number.

FR-IM-004 — Injury Recording `Critical`

If personal injury occurred: names of injured persons, nature of injuries, medical attention required, hospitalization, and facility attended. Personal injury incidents auto-generate a Critical SystemAlert to Fleet Manager and HSE Officer.

FR-IM-005 — Root Cause and Corrective Action Recording `High`

Required before closure: root cause category (Human Error, Mechanical Failure, Environmental Conditions, Procedural Violation, or Unknown) and corrective action description. Investigating HSE Officer name and closure date recorded.

FR-IM-006 — Incident Impact on Driver Score `Critical`

On incident closure, driver's incident score is updated: Near Miss −2, Minor Accident −5, Major Accident −15, Personal Injury −20, Fire or Equipment Tip-Over −25. Score recovery of +1 point per 90-day incident-free period applied by nightly batch job.

FR-IM-007 — Incident Analytics `High`

Incident analytics view: total incidents by type per site per period, incident rate per driver (incidents per 10,000 operating hours), repeat incident analysis by asset and driver, most common root cause categories, and total estimated damage cost.

B. Transit Approvals & Incident Reporting Logic

- **Transit Authorization Flow:** Open High-Resolution Sequence Diagram 1 — Transfer Workflow ↗

- **Incident Logging & Safety Verification:** Open High-Resolution Sequence Diagram 4 — Incident & Driver Safety ↗

2.8 Module PS: Project & Site Management

FR-PS-001 — Project Registration `Critical`

Fleet Manager and System Admin create project records: project code (auto-generated PRJ-[COUNTRY CODE]-YYYY-NNN), project name, client name, sector, city, country, GPS coordinates, start date, planned completion date, and project manager.

FR-PS-002 — Site Engineer Assignment to Project `High`

Multiple site engineers assignable to one project. Assignment records: employee ID, start date, and end date (null if active). Active engineers displayed on project detail page.

FR-PS-003 — Project Equipment Summary `High`

Project detail page shows: asset count by category, asset list with status, upcoming maintenance alerts, fuel consumed (current month and YTD), incident count, and predictive maintenance risk distribution (Low/Medium/High counts).

FR-PS-004 — Project Status Management `High`

Statuses: Mobilization, Active, On Hold, Demobilization, Completed. Completing a project verifies all assets have been transferred out or returned to yard. Remaining active assignments trigger a warning listing those assets; Fleet Manager confirmation required.

2.9 Module AD: Analytics Dashboard

FR-AD-001 — Executive Overview Dashboard `Critical`

Executive and Fleet Manager roles see KPIs for current month and selectable historical periods: total fleet size by status, fleet utilization rate, fleet-wide MTBF, fleet-wide MTTR, total fuel expenditure, and total maintenance expenditure. All KPI cards show percentage change vs. previous equivalent period.

FR-AD-002 — Fleet Utilization Rate Visualization `Critical`

Bar chart of utilization rate by asset category and project site for selectable periods (current week/month/last 3 months/last 12 months). Utilization = actual productive hours ÷ available hours (excluding Under Maintenance and Written Off time).

FR-AD-003 — MTBF and MTTR Trend Visualization `High`

MTBF and MTTR trend lines per asset class as a line chart. Declining MTBF highlighted in red (deteriorating fleet health). Increasing MTTR highlighted in amber (workshop capacity or parts supply issues).

FR-AD-004 — Total Cost of Ownership Per Asset `High`

TCO = amortized purchase price + fuel cost + maintenance cost (parts + labor) + insurance premium (for the period). Displayable per asset, per asset class, and per project site. Top ten highest TCO assets listed on executive dashboard.

FR-AD-005 — Fuel Consumption Heatmap `High`

Heatmap of fuel consumption per project site per week or month. Top 20% consumption sites shown in highest heat band.

FR-AD-006 — Maintenance Cost Trend `High`

Stacked area chart of monthly maintenance costs: preventive vs. corrective. Rising corrective-to-preventive ratio highlighted with chart annotation.

FR-AD-007 — Unified Expiry Alert Panel `Critical`

Consolidated alert panel for Fleet Manager: all upcoming expiries across all entities. Expiring within 30 days shown in red; within 60 days in amber. Sortable by days remaining, filterable by alert type.

FR-AD-008 — Assets Due for Service Panel `Critical`

Panel listing assets due for preventive maintenance in next 7, 14, and 30 days (based on extrapolated meter reading progression). Visible to Fleet Manager and Workshop Supervisor. Directly actionable by creating a job card from the list.

A. Machine Learning Pipeline & Asynchronous Sensor Streams

The raw edge telemetry ingestion architecture, features extraction loops, and live UI push communication channels are structured down below:

Click to Open High-Resolution Analytics Pipeline DFD ↗

2.10 Module IL: Intelligence Layer

FR-IL-001 — Predictive Maintenance Scoring `Critical`

Binary classification ML model producing breakdown probability score (0–100) per active asset. Input features: asset age, cumulative hours, hours since last service, breakdowns in last 90/365 days, fuel anomaly count, overdue maintenance flag, operating environment, and asset category. Retrained on a defined schedule.

FR-IL-002 — Predictive Score Display and Action Routing `Critical`

Score and risk category displayed on asset detail page and Fleet Manager dashboard. Risk bands: Low (0–39), Medium (40–69), High (70–100). High risk assets auto-generate a SystemAlert to Fleet Manager and Workshop Supervisor with recommended action.

FR-IL-003 — Fuel Consumption Forecasting `High`

Regression ML model forecasting total fuel volume required per project site for next 7 and 30 days. Inputs: active equipment count by sub-type, historical daily consumption per sub-type, day-of-week patterns, and project activity intensity (1–5 scale, manually set by Project Manager). Displayed on site detail page and executive dashboard.

FR-IL-004 — Driver Behavior Scoring Model `High`

Classification model assigning Low/Medium/High risk per driver. Composite score: incident history (40%), fuel overconsumption frequency (30%), breakdown attribution (20%), license/medical compliance (10%). Recomputed nightly. Score weights configurable by System Admin.

FR-IL-005 — Replacement Alert Engine `High`

Rule-based engine monitoring cumulative corrective maintenance cost as a percentage of current book value. Alert at configurable threshold (default 60%): ReplacementAlert record routed to Fleet Manager with asset details, book value, cumulative repair cost, ratio, and suggested action. Second alert at 80%. Runs nightly.

FR-IL-006 — Model Evaluation Reporting `High`

Stored evaluation metrics per trained model version. Predictive maintenance: accuracy, precision, recall, F1-score, ROC-AUC. Fuel forecasting: MAE, RMSE, R^2. Driver behavior: accuracy, precision, recall, F1-score per class. Accessible to System Admin and Fleet Manager.

B. Asynchronous Job Scheduling & Inference Processing

The automated data pull cycles, training executions, predictive RUL algorithms, and exception threshold triggers execute in this timeline:

- **Cron & Inference Pipeline Flow:** Open High-Resolution Sequence Diagram 5 — Nightly AI Batch ↗

2.11 Module SA: System Administration

FR-SA-001 — User Account Management `Critical`

System Admin creates, edits, deactivates, and reactivates user accounts. Each account is linked to an Employee record with exactly one Role. Deactivated accounts cannot log in but all historical records are retained for audit integrity.

FR-SA-002 — Role-Based Access Control Enforcement `Critical`

RBAC enforced at the API layer on every endpoint. Front-end UI suppresses navigation items, buttons, and fields the user's role cannot access. Both layers are mandatory — front-end suppression alone is not sufficient.

FR-SA-003 — Audit Log Access `High`

Read-only audit log viewer for Fleet Manager and System Admin. Filterable by: table name, record ID, operation type, user, and date range. Non-editable and non-deletable by any role.

FR-SA-004 — System Configuration Management `High`

Configuration panel (System Admin only) for: fuel anomaly deviation threshold, maintenance overdue warning/critical thresholds, replacement alert trigger threshold, driver high-risk score threshold, alert notification lead times, and driver behavior score weights.

FR-SA-005 — Lookup Table Management `Medium`

System Admin can add, edit, and soft-delete records in lookup tables without a code deployment: AssetCategory, AssetSubType, FaultCategory, IncidentType, TransferReason, ServiceType, LicenseCategory, TrainingType, DocumentType, and VendorType.

Section 3 — Non-Functional Requirements

3.1 Performance Requirements

NFR-P-001 — Dashboard Load Time

The executive dashboard and fleet manager dashboard shall load and render all KPI widgets within 3 seconds for a dataset of up to 5,000 assets and 3 years of operational history, measured on a standard broadband connection of 10 Mbps or higher. KPI values shall be served from the pre-computed AssetKPISnapshot table and shall not be calculated dynamically from raw log tables on each page load.

NFR-P-002 — Form Submission Response Time

All data entry form submissions (fuel log, job card, incident report, transfer request) shall return a success or error response within 1 second of the user submitting the form, under a concurrent user load of up to 200 simultaneous active users.

NFR-P-003 — Search Response Time

All search and filter operations on list views shall return results within 2 seconds for datasets up to 10,000 records per table, achieved through appropriate database indexing.

NFR-P-004 — ML Model Scoring Batch Job

The nightly batch job (predictive maintenance scores, driver behavior scores, fuel forecasts, KPI snapshots, replacement alerts) shall complete its full run within a 4-hour window (00:00–04:00 Gulf Standard Time) without impacting daytime application performance.

NFR-P-005 — File Upload Performance

Document and image file uploads up to 10 MB shall complete within 5 seconds on a 10 Mbps connection. Files shall be validated for type and size before upload begins.

3.2 Scalability Requirements

NFR-SC-001 — Asset Scale

The database schema and application architecture shall support growth from 50 assets to a minimum of 5,000 assets without schema migration or architectural change.

NFR-SC-002 — Operational Log Volume

Designed to handle estimated annual volumes without performance degradation: 73,000 fuel log entries, 18,000 utilization log entries, 2,400 maintenance job cards, 600 breakdown log entries, and 120 transfer requests. High-volume tables shall have composite indexes on (asset_id, created_at) as a minimum.

NFR-SC-003 — Concurrent User Load

Supports a minimum of 200 concurrent authenticated users without response time degradation beyond NFR-P-002 limits. The API layer is stateless to enable horizontal scaling via load balancer if required.

3.3 Security Requirements

NFR-SEC-001 — Authentication

All access requires authentication via username and password. Passwords are hashed using bcrypt with a minimum work factor of 12 before storage. No plaintext passwords stored at any point.

NFR-SEC-002 — Session Management

Authentication uses JWTs with configurable expiry (default 8 hours, aligned to a standard work shift). Refresh tokens with a maximum lifetime of 7 days. Logout invalidates the refresh token server-side.

NFR-SEC-003 — API Authorization

Every API endpoint validates the JWT and the user's role before processing any request. Authorization failures return HTTP 403 with no information about the resource, preventing information leakage.

NFR-SEC-004 — Financial Data Access Control

Fields including purchase price, depreciation values, current book value, insurance premium amounts, and TCO figures are returned only to Fleet Manager, Finance, or Executive roles. API responses for other roles omit these fields entirely — not merely hidden on the front end.

NFR-SEC-005 — Data Transmission Security

All data transmitted between client and server is encrypted using HTTPS with TLS 1.2 or higher. HTTP connections redirected to HTTPS. No sensitive data transmitted in URL query parameters.

NFR-SEC-006 — Input Validation and Injection Prevention

All user inputs validated and sanitized at the API layer before any database interaction. Parameterized queries or ORM with parameterized query support used exclusively. Raw SQL string concatenation is prohibited.

NFR-SEC-007 — Audit Trail Non-Repudiation

The AuditLog table is written to by the database application user with INSERT-only permissions. No application-layer user, including System Admin, has UPDATE or DELETE permissions on AuditLog. This ensures complete non-repudiation.

3.4 Availability and Reliability Requirements

NFR-AV-001 — System Uptime

Minimum uptime of 99.5% during operational hours (06:00–22:00 Gulf Standard Time, all days). Maximum allowable downtime: approximately 87 minutes per month during operational hours.

NFR-AV-002 — Scheduled Maintenance Window

Maintenance requiring downtime is scheduled within 00:00–04:00 GST on Fridays only. Users receive in-app notification 24 hours before any scheduled maintenance window.

NFR-AV-003 — Data Backup

Full database backup daily at 02:00 GST; incremental transaction log backups every 4 hours. Backups retained for a minimum of 90 days. Backup restoration test performed at minimum once per month.

NFR-AV-004 — Graceful Error Handling

The system shall never display raw database error messages, stack traces, or internal system paths to end users. All errors are caught and displayed as user-friendly messages with an error reference code that maps to the server-side log.

3.5 Usability Requirements

NFR-US-001 — Mobile Responsiveness

All data entry forms and list views shall be fully functional on a mobile browser viewport of minimum 375 pixels width. Navigation via collapsible mobile menu. Touch targets minimum 44 pixels in height.

NFR-US-002 — Localization

The UI supports English and Arabic, selectable per user. Arabic mode renders in right-to-left layout. Dates displayable in Gregorian or Hijri calendar format per user preference. Currency supports QAR, AED, SAR, OMR, BHD, KWD, and USD.

NFR-US-003 — Accessibility

Meets WCAG 2.1 Level AA for all core functional pages: minimum 4.5:1 color contrast for normal text, full keyboard navigability, and descriptive alt text for all non-decorative images and chart visualizations.

NFR-US-004 — Non-Technical User Operability

A Site Engineer with no prior EMS training but basic computer literacy shall be able to successfully complete a fuel log entry, a breakdown report, and a transfer request submission within 15 minutes of first access, based on the interface alone. Validated during UAT.

3.6 Maintainability Requirements

NFR-MT-001 — Code Documentation

All API endpoint handlers, database model definitions, and ML training scripts shall include inline documentation comments. A README at the repository root shall provide complete local development setup instructions.

NFR-MT-002 — Environment Configuration

All environment-specific configuration values (database connection string, JWT secret, file storage path, API keys, ML model paths) stored in environment variables and never hardcoded. A documented `.env.example` file lists all required variables with descriptions.

NFR-MT-003 — Modular Architecture

The back-end API shall be organized into modules corresponding to the functional modules in Section 2. Each module contains its own routes, controllers, and service layer. Cross-module dependencies are minimized and explicitly documented.

3.7 Regulatory Compliance Requirements

NFR-RC-001 — Gulf Vehicle Regulation Compliance

No transfer dispatch permitted for a road-registered asset with expired Gulf Registration or expired insurance. Fleet Manager override is permitted only with a mandatory logged justification.

NFR-RC-002 — Lifting Equipment Certification Compliance

No Lifting and Heavy Shifting Equipment asset can be set to Active status if its most recent EquipmentCertification has expired. This is a hard block with no override available to any role.

NFR-RC-003 — Incident Documentation Compliance

A Major Accident incident report cannot be closed without a police report number entered, as required under GCC traffic regulations.

NFR-RC-004 — Data Retention

All operational records (fuel logs, job cards, breakdown logs, incident reports, transfer records) shall be retained for a minimum of 7 years from creation date, in compliance with GCC commercial and regulatory record-keeping requirements. Soft deletes are the only permitted deletion mechanism on these tables.

Section 4 — System Architecture

4.1 Architectural Style

The EMS is built on a three-tier client-server architecture with a separately deployed Python-based machine learning service. The three tiers are:

- **Presentation Tier** — Next.js React front end
- **Application Tier** — Node.js Express REST API
- **Data Tier** — PostgreSQL relational database

The ML service (Python FastAPI) operates as a sidecar service alongside the application tier, called by scheduled batch jobs and not directly accessible from the client browser.

This architecture was chosen over alternatives (monolithic MVC or microservices) for the following reasons: sufficient separation of concerns for a twelve-week project without microservice orchestration overhead; independent evolution of front end, back end, and data layer; horizontal scaling support for the API tier; and correct runtime separation between Python (ML/data science) and Node.js (high-concurrency I/O).

Figure 4.1: Component Infrastructure, API Layer, and ML Analytics Pipelines

---

4.2 Presentation Tier

The front end is built with **Next.js 14** (App Router), **React 18**, and **Tailwind CSS**. Next.js provides server-side rendering for initial page loads, client-side navigation between pages, and API route proxying (avoiding CORS complexity and hiding the back-end API URL from browser clients).

Page Groups:

- Authentication: Login
- Dashboards: Executive Dashboard, Fleet Manager Dashboard
- Asset pages: Asset List, Asset Detail, Asset Registration Form, Asset Edit Form
- Driver pages: Driver List, Driver Detail, Driver Registration
- Fuel pages: Fuel Log Entry, Fuel Log History, Site Tank Management
- Maintenance pages: Job Card List, Job Card Detail, Job Card Creation, Maintenance Schedule View
- Transfer pages: Transfer Request Form, Approval Queue, Transfer Detail, Transfer History
- Incident pages: Incident Report Form, Incident List, Incident Detail
- Project pages: Project List, Project Detail, Project Registration
- Analytics pages: Utilization Dashboard, Cost Dashboard, Fuel Heatmap, MTBF & MTTR Trends
- Intelligence pages: Predictive Maintenance Scores, Fuel Forecast, Driver Behavior Scores, Replacement Alerts
- Administration pages: User Management, System Configuration, Lookup Table Management, Audit Log Viewer

State Management: React Context for authentication state (user identity and role); React Query (TanStack Query) for server state management — automatic caching, background refetching, and loading/error state handling for all API calls.

4.3 Application Tier

The back-end API is built with **Node.js** and **Express.js**, structured as a RESTful API following standard HTTP conventions: GET (retrieval), POST (creation), PUT (full update), PATCH (partial update), DELETE (soft-delete).

Layer Structure (outermost to innermost):

1. **Routes Layer** — defines URL paths, maps to controller functions
2. **Middleware Layer** — JWT authentication validation, role authorization, input validation (express-validator), and request logging
3. **Controller Layer** — extracts validated inputs, calls service functions, formats HTTP responses
4. **Service Layer** — contains business logic: derived value calculations, business rule enforcement (maintenance block on transfer, fuel tank capacity validation, license eligibility check), and multi-table transaction orchestration
5. **Data Access Layer** — Prisma ORM for type-safe, parameterized PostgreSQL queries

Route Modules: `/api/assets`, `/api/drivers`, `/api/fuel`, `/api/maintenance`, `/api/transfers`, `/api/incidents`, `/api/projects`, `/api/analytics`, `/api/intelligence`, `/api/admin`

Each module contains its own routes file, controller file, and service file.

4.4 Data Tier

The data tier is **PostgreSQL 15**. Selected for:

- Native JSONB column support (used for old/new value snapshots in AuditLog)
- Excellent support for complex analytical queries (dashboard and ML feature extraction)
- Strong foreign key constraint enforcement matching the normalized schema design
- Row-level security support as a future enhancement layer
- Widespread availability on AWS RDS, Azure Database for PostgreSQL, and Google Cloud SQL

The database is accessed exclusively through the Prisma ORM from the Node.js application tier. Direct database connections from the front end or client browsers are prohibited by network configuration.

4.5 Machine Learning Service

The ML service is a **Python 3.11** application built with **FastAPI**. It exposes the following internal endpoints (callable only from the Node.js API server):

- `POST /ml/predict-maintenance` — accepts batch of asset feature vectors, returns breakdown probability scores
- `POST /ml/forecast-fuel` — accepts site feature vectors, returns fuel volume forecasts
- `POST /ml/score-drivers` — accepts batch of driver feature data, returns driver behavior scores

These endpoints are **not exposed to the public internet**.

Libraries: scikit-learn (Random Forest for predictive maintenance and driver behavior classification); XGBoost (fuel forecasting regression). Trained models serialized using joblib, stored in the file system, and loaded by FastAPI at startup.

4.6 Scheduled Batch Jobs

A Node.js scheduler (node-cron) executes the following nightly tasks at 00:00 GST in order:

1. Extract feature vectors for all active assets → call `/ml/predict-maintenance` → write results to PredictiveMaintenanceScore table
2. Extract feature vectors for all active projects → call `/ml/forecast-fuel` → write results to FuelForecast table
3. Extract driver behavior feature data → call `/ml/score-drivers` → write results to DriverBehaviorScore table
4. Compute asset KPI snapshots (MTBF, MTTR, utilization rate, TCO) from raw log tables → write to AssetKPISnapshot table
5. Run the replacement alert engine across all assets → generate ReplacementAlert records where thresholds are crossed
6. Check all expiry dates (registrations, insurance, certifications, licenses) → generate SystemAlert records for items entering alert windows

---

4.7 Authentication Architecture

Stateless JWT authentication approach:

- On successful login: server issues an **access token** (8-hour expiry) and a **refresh token** (7-day expiry)
- Access token stored in **browser memory** (not localStorage — prevents XSS token theft)
- Refresh token stored in an **httpOnly, Secure, SameSite=Strict cookie** (inaccessible to JavaScript)
- Next.js front end proxies all API calls through its own server, attaching the access token from memory — preventing the raw token from being accessible in browser developer tools in production

---

4.8 File Storage Architecture

Document and image attachments are stored in the **file system or cloud object storage** (AWS S3 or equivalent) — not in the database. The DocumentAttachment table stores only metadata: filename, file path/URL, file type, file size, uploaded by, and uploaded at.

Direct file URLs are **never returned to the client**. Instead, the API generates short-lived **signed URLs** for file access, preventing unauthorized hotlinking of sensitive documents such as insurance certificates or purchase invoices.

4.9 System Class and Structural Models

The low-level object-oriented architecture, data representations, and service layer methods are split into two core blueprints:

A. Data Domain Entity Mapping

This structural blueprint maps out the internal programmatic objects, properties, and relationships reflecting our core database schema:

- **Domain Models Architecture:** View UML Class Diagram Part A — Domain ↗

B. Business Logic & Controller Layer Mapping

This structural blueprint maps out the execution classes, API controllers, dependency injections, and background processor service methods:

- **Service Tier Architecture:** View UML Class Diagram Part B — Services ↗

Section 5 — Data Management

5.1 Data Architecture Principles

The EMS data architecture is governed by four principles:

1. **Single source of truth** — Every data point is recorded once, in one place, and all other parts of the system reference it rather than duplicate it. The Asset record is the single source of truth for asset identity. The Employee record is the single source of truth for user identity.
2. **Append-only operational logs** — Fuel logs, utilization logs, breakdown logs, and incident reports are never updated after creation. Corrections are made by creating a new record referencing the original with a correction flag, preserving the original entry for audit purposes.
3. **Soft deletes only** — No operational record is ever physically deleted from the database. Deletion means setting a status field to Archived or Inactive, retaining the full record with its history intact.
4. **Separation of OLTP and OLAP concerns** — Real-time data entry (OLTP) operates against fully normalized tables. Dashboard and analytics queries (OLAP) operate against pre-computed snapshot tables populated by the nightly batch job, preventing heavy analytical queries from impacting operational data entry performance.

5.2 Database Normalization Standard

All master registry tables and lookup tables are normalized to **Third Normal Form (3NF)**:

- Every table has a primary key (1NF)
- Every non-key column depends on the whole primary key, not a subset (2NF)
- Every non-key column depends only on the primary key, not on any other non-key column (3NF)

Design decisions driven by 3NF:

- `EngineSpecification` is a separate table from `Asset` (engine attributes depend on the engine, not the asset number)
- `PurchaseRecord` and `InsurancePolicy` are separate tables from `Asset` (financial attributes depend on the financial transaction, not the asset)
- `Vendor` is a separate table from `PurchaseRecord`, `InsurancePolicy`, and `MaintenanceJobCard` (vendor attributes depend on the vendor identity, not the referencing transaction)

Intentional denormalization exists only in analytics snapshot tables (`AssetKPISnapshot`, `PredictiveMaintenanceScore`, `FuelForecast`, `DriverBehaviorScore`) where pre-computed values serve the dashboard. These are documented as deliberate, justified departures from 3NF.

5.3 Primary Key Strategy

All primary keys use **UUID version 4** rather than auto-incrementing integers. Reasons:

- UUIDs are globally unique and safe to generate at the application layer before database insertion (enabling optimistic UI updates)
- They prevent enumeration attacks (an attacker cannot simply increment an ID to access another record)
- They are safe to use across a distributed or sharded system

UUID generation happens in the application tier using the `uuid` library before the INSERT statement, not at the database layer.

5.4 Indexing Strategy

Beyond primary key indexes, the following indexes are created to support the most frequent query patterns:

- **FuelLog**: Composite index on `(asset_id, logged_at DESC)`; composite index on `(project_id, logged_at DESC)`
- **MaintenanceJobCard**: Composite index on `(asset_id, opened_at DESC)`; index on `(status)`
- **BreakdownLog**: Composite index on `(asset_id, occurred_at DESC)`; composite index on `(driver_id, occurred_at DESC)`
- **AssetSiteAssignment**: Composite index on `(asset_id, assigned_at DESC)`; index on `(project_id)` with partial index on `(removed_at IS NULL)` for current assignments
- **TransferRequest**: Index on `(status)` to support the approval queue
- **SystemAlert**: Composite index on `(entity_type, entity_id, is_acknowledged)` for the unified alert panel
- **AuditLog**: Composite index on `(table_name, record_id, performed_at DESC)` for audit trail queries

---

5.5 Data Validation Rules

The following validation rules are enforced at both the **application layer** (API middleware) and the **database layer** (CHECK constraints and NOT NULL constraints) for dual-layer data integrity:

- Asset number format must match the pattern `EQ-[0-9]{4}-[0-9]{4}`
- Engine serial number must be unique across all EngineSpecification records (UNIQUE constraint)
- Chassis serial number must be unique across all EngineSpecification records (UNIQUE constraint)
- Fuel quantity must be greater than zero and must not exceed the tank capacity in the asset's EngineSpecification record
- Meter reading on fuel log must be greater than or equal to the previous fuel log meter reading for the same asset (prevents retroactive entries)
- Job card cannot be closed without at least one labor entry or one parts entry
- Transfer departure date must be on or after the approval date (database CHECK constraint)
- Incident report date must not be in the future (database CHECK constraint)

- Insurance coverage end date must be after coverage start date (database CHECK constraint)

5.6 Data Retention Policy

Record Type	Retention Period
Financial records (PurchaseRecord, InsurancePolicy, cost data)	Permanently retained
Operational logs (FuelLog, UtilizationLog, AssetOperatorAssignment)	Minimum 7 years from creation
Incident reports and transfer records	Minimum 10 years from closure date
Maintenance records	Lifetime of asset + 7 years after write-off
Analytics snapshot tables	3 years rolling history; older records archived, not deleted
AuditLog records	Permanently retained; never truncated or partitioned out

---

5.7 Seed Data Strategy

The system will be seeded with the following data to support development, testing, and AI model training:

- **50 baseline assets** across 5 categories with realistic engine serial numbers, chassis numbers, purchase dates (2018–2023), and OEM-accurate service intervals
- **5 project records** representing active construction sites across two Gulf countries
- **20 driver records** with varied license categories, nationalities, and experience levels
- **12 months of synthetic operational data** programmatically generated:
  - Daily fuel logs for all 50 assets (365 days × 50 assets with realistic consumption variation)
  - Monthly preventive maintenance job cards (~200 records)
  - 45 randomly distributed breakdown events biased toward older assets and poor maintenance adherence
  - 12 incident reports of mixed severity types
  - 25 equipment transfers between the five project sites
  - Daily utilization logs for all assets

The synthetic data generation script is a standalone Python file committed to the repository, allowing the database to be reset and reseeded at any time during development.

Section 6 — User Documentation Requirements

6.1 In-Application Help

The system shall provide contextual help within the application interface for every data entry form. Each form field that is not immediately self-explanatory shall have an information icon adjacent to it. Clicking the icon shall display a tooltip or modal explaining:

- What the field represents
- Why it is required
- A valid example value

This is particularly important for fields such as Gulf Registration Certificate Number, Carnet de Passage, Engine Serial Number, and OEM Service Interval formats.

6.2 User Role Guides

A separate user guide document shall be prepared for each of the following role groups, covering only the modules and workflows relevant to that role:

- **Fleet Manager User Guide**
- **Site Engineer User Guide**
- **Workshop Mechanic and Supervisor User Guide**
- **HSE Officer User Guide**
- **System Administrator Guide**

Each guide shall include step-by-step screenshots of all workflows the role is expected to perform, annotated with numbered callouts.

6.3 System Administrator Guide

The System Administrator Guide shall cover:

- Creating and managing user accounts
- Configuring system thresholds and alert parameters
- Managing lookup tables
- Accessing the audit log viewer
- Understanding the nightly batch job schedule and how to check its execution logs
- Performing a database backup restoration test
- Procedures for adding new asset categories or sub-types

6.4 API Documentation

All API endpoints shall be documented using the **OpenAPI 3.0 specification**, auto-generated from code annotations using the `swagger-jsdoc` library. The Swagger UI shall be accessible at `/api/docs` in the

development environment (disabled in production or protected behind admin authentication).

Documentation shall include:

- Endpoint URL and HTTP method
- Required request headers
- Request body schema with field descriptions and validation rules
- Response schema for success and error cases
- Example request and response payloads

6.5 Data Dictionary

The complete Data Dictionary covering all 45 entities, every table column, data types, constraints, descriptions, and example values constitutes a standalone Phase 1 deliverable document and is incorporated by reference into this SRS.

The global database layer is broken into 5 isolated structural schemas. Review the structural design, raw DBML code, and field indexes below:

1. Asset Master Domain Schema

- **Visual Data Model:** Open High-Resolution Interactive ERD ↗
- **Schema Definition Script:** Raw DBML Code Script ↗
- **Column Index Definitions:** Comprehensive Field Data Dictionary ↗

2. Driver amd People Domain Schema

- **Visual Data Model:** Open High-Resolution Interactive ERD ↗
- **Schema Definition Script:** Raw DBML Code Script ↗
- **Column Index Definitions:** Comprehensive Field Data Dictionary ↗

3. Operations Domain Schema

- **Visual Data Model:** Open High-Resolution Interactive ERD ↗
- **Schema Definition Script:** Raw DBML Code Script ↗
- **Column Index Definitions:** Comprehensive Field Data Dictionary ↗

4. Maintenance & Transfer Domain Schema

- **Visual Data Model:** Open High-Resolution Interactive ERD ↗
- **Schema Definition Script:** Raw DBML Code Script ↗
- **Column Index Definitions:** Comprehensive Field Data Dictionary ↗

5. Analytics & Audit Domain Schema

- **Visual Data Model:** Open High-Resolution Interactive ERD ↗
- **Schema Definition Script:** Raw DBML Code Script ↗
- **Column Index Definitions:** Comprehensive Field Data Dictionary ↗

Section 7 — Testing Requirements

7.1 Testing Strategy Overview

Testing for the EMS shall follow a **pyramid structure**: the most tests at the unit level, fewer at the integration level, and the fewest at the end-to-end level.

Four distinct testing phases are defined:

1. **Unit Testing** — Developer-executed during development
2. **Integration Testing** — Developer-executed before each phase completion
3. **System Testing** — Developer-executed before UAT
4. **User Acceptance Testing (UAT)** — Mentor-executed at project completion

---

7.2 Unit Testing Requirements

TR-UT-001

All service layer business logic functions shall have unit tests covering the happy path, all validation failure paths, and all edge case paths identified in the requirements. Target minimum code coverage for service layer files: **80%**.

TR-UT-002

The following specific business logic units shall have dedicated unit tests:

- **Fuel tank capacity validation** — quantity within limit passes; at exactly limit passes; exceeding limit fails; zero quantity fails
- **Meter reading progression validation** — reading greater than previous passes; equal passes; less than previous fails
- **Transfer compliance check** — all documents valid passes; registration expired fails; insurance expired fails; driver license expired fails
- **Maintenance overdue calculation** — under interval passes; at 5% over produces warning; at 10% over produces critical; exactly at interval passes
- **Replacement alert threshold calculation** — ratio below threshold produces no alert; at threshold produces first alert; at secondary threshold produces second alert

TR-UT-003

All ML model feature extraction functions shall have unit tests validating correct feature vector construction from sample database records.

7.3 Integration Testing Requirements

TR-IT-001

All API endpoints shall have integration tests executed against a test database (separate from development and production databases). Tests shall cover:

- Valid authenticated request returns correct data
- Unauthenticated request returns HTTP 401
- Request by insufficient role returns HTTP 403
- Invalid input returns HTTP 422 with field-level error messages
- Creation endpoints return the newly created record with all fields populated

TR-IT-002

The following multi-step workflow integration tests shall be executed:

- **Complete transfer workflow** — request submission → approval → dispatch → arrival confirmation, including verification that asset site assignment is updated on arrival
- **Complete job card workflow** — creation → parts addition → labor addition → supervisor closure, including verification that next service schedule is updated on closure
- **Complete breakdown workflow** — breakdown log creation → job card creation → closure, including verification that MTTR is calculated correctly

TR-IT-003

The nightly batch job shall be tested against a test dataset where expected output values are pre-calculated manually, and the batch job output shall be verified to match expected values within acceptable tolerance.

7.4 System Testing Requirements

TR-ST-001 — RBAC Boundary Testing

For each role, a test script shall systematically attempt to access every endpoint that role is not permitted to access and verify that HTTP 403 is returned in every case. The test matrix covers all 5 roles against all 50+ API endpoint groups.

TR-ST-002 — Data Integrity Testing

Test cases shall verify that all database-level constraints are enforced: foreign key violations rejected, NOT NULL violations rejected, UNIQUE constraint violations rejected, and CHECK constraint violations rejected. Each constraint shall have at least one test case.

TR-ST-003 — Edge Case Scenario Testing

The following specific edge case scenarios shall each have a documented test case with steps, expected result, and pass/fail recorded:

- Asset breakdown during transit — verify transfer status updates to In Transit — Breakdown and fleet manager alert is generated

- Fuel entry exceeding tank capacity — verify system blocks entry and displays correct tank capacity in error message
- Transfer dispatch while asset is under maintenance — verify system blocks dispatch and displays open job card reference
- Repeat failure within repair warranty period — verify warranty claim alert is generated
- Expired license driver assignment to executive vehicle — verify warning or block per configuration
- Lifting equipment certification expiry — verify hard block on Active status assignment
- Project completion with remaining asset assignments — verify warning listing unassigned assets

TR-ST-004 — Performance Testing

Load testing using k6 or Apache JMeter simulating 200 concurrent users performing mixed operations (60% read/dashboard operations, 30% form submissions, 10% search operations). Test shall verify NFR-P-001 and NFR-P-002 are met under this load.

7.5 User Acceptance Testing Requirements

TR-UAT-001 — UAT Scope

UAT conducted by the three mentors acting as representative stakeholders:

- Mentor 1 → Fleet Manager role
- Mentor 2 → Site Engineer and Workshop Supervisor role
- Mentor 3 → HSE Officer and Executive role

TR-UAT-002 — UAT Test Scripts

A formal UAT test script document shall be prepared with step-by-step instructions for each test scenario and clearly stated expected outcomes. The script shall cover every functional requirement marked **Critical** or **High** priority in Section 2.

TR-UAT-003 — UAT Sign-Off

UAT is considered passed when:

- All Critical and High priority test cases pass
- No more than 20% of Medium priority test cases have defects

Any failed Critical test case is an automatic UAT failure requiring remediation and re-test before final presentation.

Section 8 — Acceptance Criteria

8.1 Database and Schema Criteria

AC-DB-001

All 45 entities defined in the Entity List deliverable are implemented as database tables with all columns, data types, NOT NULL constraints, UNIQUE constraints, and foreign key relationships correctly configured. Zero tables may be missing from the schema at UAT.

AC-DB-002

All 50 baseline assets are registered in the system with complete data. No mandatory field on any of the 50 asset records may contain a null value at the time of UAT.

AC-DB-003

The synthetic operational dataset covering 12 months of fuel logs, maintenance records, breakdown events, incidents, and transfers is present in the database and passes referential integrity checks (no orphaned foreign keys).

8.2 Application Functionality Criteria

AC-APP-001

A Fleet Manager can complete the end-to-end equipment transfer workflow from transfer request submission through final arrival confirmation in under **5 minutes** of total cumulative user interaction time, excluding system response delays.

AC-APP-002

The system correctly enforces all five critical business rule blocks — each verified by specific UAT test cases:

1. Transfer dispatch while asset is under maintenance
2. Transfer dispatch with expired Gulf registration
3. Fuel quantity exceeding tank capacity
4. Lifting equipment without valid certification set to Active
5. Expired-license driver assigned to a vehicle

AC-APP-003

RBAC is enforced such that a logged-in Workshop Mechanic user receives HTTP 403 responses when attempting to access financial data endpoints, transfer approval endpoints, and executive dashboard data endpoints — verified by the RBAC boundary test (TR-ST-001).

8.3 Dashboard Criteria

AC-DASH-001

The executive dashboard displays all six core KPIs — Fleet Utilization Rate, MTBF, MTTR, TCO, Fuel Spend Trend, Incident Rate — populated with the synthetic 12-month dataset and loads fully within **3 seconds** on the UAT environment connection.

AC-DASH-002

The unified expiry alert panel correctly identifies and displays all expiring items within the 90-day window from the synthetic dataset with correct color coding (red within 30 days, amber within 60 days).

8.4 Machine Learning Criteria

AC-ML-001

The predictive maintenance classification model achieves a minimum **F1-Score of 0.75** on the held-out test partition of the synthetic dataset. The evaluation report displaying this metric is accessible in the system's model evaluation page.

AC-ML-002

The fuel consumption forecasting regression model achieves a minimum **R^2 of 0.80** on weekly site-level fuel data from the synthetic dataset.

AC-ML-003

The driver behavior classification model achieves a minimum overall **accuracy of 80%** on the test partition of the synthetic driver behavior dataset.

8.5 Code Quality Criteria

AC-CQ-001

The complete source code is committed to a Git repository with a minimum of **50 meaningful commits** demonstrating incremental development progress. Commit messages shall follow conventional commit format (`feat:` , `fix:` , `docs:` , `test:` , `refactor:` prefixes).

AC-CQ-002

A `README.md` file at the repository root provides complete local development setup instructions that a developer unfamiliar with the project can follow to get the application running from scratch in under **30 minutes**.

AC-CQ-003

Unit test code coverage for the service layer is at minimum **80%** as measured by the project's test coverage tool.

Section 9 — Project Timeline (WBS)

9.1 Overview

The project spans **12 weeks** divided into five phases as defined in the original project assignment. The WBS below expands each phase into specific tasks with estimated effort and deliverables.

Phase 1: Weeks 1-2 — Requirements, Design & Architecture

Gate Condition: Mentor sign-off on SRS and ERD before any application code is written.

Week 1

Task 1.1 — Stakeholder Analysis and Requirements Gathering

Document all stakeholder groups, their data inputs and outputs, and operational pain points.

Output: Stakeholder Analysis section of SRS. Effort: 1 day.

Task 1.2 — Functional and Non-Functional Requirements Documentation

Complete FR and NFR sections of SRS with unique IDs and priorities.

Output: FR and NFR sections of SRS. Effort: 1.5 days.

Task 1.3 — Entity List and Relationship Definition

Define all entities, all relationships, and cardinality notation.

Output: Deliverable 1 — Entity List and Relationships document. Effort: 1 day.

Task 1.4 — Data Dictionary Preparation

Define every table and every column with type, constraints, and examples.

Output: Deliverable 2 — Data Dictionary. Effort: 1.5 days.

Week 2

Task 2.1 — ERD Diagram Creation

Produce crow's foot notation ERD in draw.io or dbdiagram.io covering all 45 entities.

Output: ERD Diagram deliverable. Effort: 1 day.

Task 2.2 — System Architecture Diagram

Produce three-tier architecture diagram showing all components and data flow.

Output: System Architecture Diagram. Effort: 0.5 days.

Task 2.3 — UML Diagrams

Produce Use Case Diagram, Class Diagram, Sequence Diagrams (Transfer and Maintenance workflows), State Machine Diagrams (Asset, Job Card, and Transfer lifecycles).

Output: UML Diagrams deliverable. Effort: 1.5 days.

Task 2.4 — Context and Data Flow Diagrams

Produce Level 0 Context Diagram and Level 1 DFD for each major process.

Output: Context and DFD deliverable. Effort: 0.5 days.

Task 2.5 — UI Wireframes

Produce low-fidelity wireframes for all 20+ screens.

Output: Wireframes deliverable. Effort: 0.5 days.

Task 2.6 — SRS Final Compilation and Mentor Review

Compile all sections into final SRS document, submit for mentor review and sign-off.

Output: SRS v1.0 submitted for review. Effort: 0.5 days.

Phase 2: Weeks 3-5 — Core Web Application Development

Gate Condition: All CRUD operations functional, all business rule blocks verified, RBAC enforced end-to-end.

Week 3

Task 3.1 — Development Environment Setup

Initialize Next.js project, Express API project, PostgreSQL database, and Prisma ORM. Configure environment variables, ESLint, Prettier, and Git repository with branch strategy. Effort: 0.5 days.

Task 3.2 — Database Schema Implementation

Translate ERD into Prisma schema file, run initial migration to create all 45 tables with all constraints and indexes. Seed lookup tables. Effort: 1.5 days.

Task 3.3 — Authentication Implementation

Build login endpoint with bcrypt hashing, JWT access token and refresh token issuance, JWT validation middleware, and role authorization middleware. Build front-end login page with JWT storage in memory. Effort: 1 day.

Task 3.4 — Asset Registry Module — Back End

Build all Asset API endpoints (CRUD for Asset, EngineSpecification, GulfRegistration, PurchaseRecord, InsurancePolicy, TyreRecord, BatteryRecord, EquipmentCertification). Implement all validation and business rule enforcement at service layer. Effort: 2 days.

Week 4

Task 4.1 — Asset Registry Module — Front End

Build Asset List page, Asset Detail page with all tabbed sections, Asset Registration multi-step form, and Document Attachment upload functionality. Effort: 2 days.

Task 4.2 — Driver Registry Module — Back End and Front End

Build Driver API endpoints and front-end Driver List and Detail pages with license validation and training record management. Effort: 1 day.

Task 4.3 — Fuel Management Module — Back End and Front End

Build Fuel Log API with tank capacity validation and anomaly detection. Build Fuel Log Entry form, Fuel History view, and Site Tank Management pages. Effort: 1.5 days.

Task 4.4 — Project and Site Module — Back End and Front End

Build Project CRUD API and Project Registration, Project List, and Project Detail pages. Effort: 0.5 days.

Week 5

Task 5.1 — Maintenance Management Module — Back End

Build Job Card API with full workflow state machine, Parts and Labor line item APIs, Preventive Schedule generation and overdue detection logic. Effort: 1.5 days.

Task 5.2 — Maintenance Management Module — Front End

Build Job Card List, Job Card Creation form, Job Card Detail with parts and labor entry, Maintenance Schedule view. Effort: 1.5 days.

Task 5.3 — Equipment Transfer Module — Back End and Front End

Build Transfer Request API with compliance check, approval workflow, state machine, and inspection records. Build Transfer Request form, Approval Queue, and Transfer Detail pages. Effort: 1.5 days.

Task 5.4 — Incident Management Module — Back End and Front End

Build Incident Report API and Incident Report form with media upload. Effort: 0.5 days.

Phase 3: Weeks 6-7 — Analytics and Reporting

Gate Condition: Dashboard fully populated, all six core KPIs visible and accurate against synthetic data, loading within 3 seconds.

Week 6

Task 6.1 — KPI Computation Batch Job

Implement the nightly Node.js batch job that computes and stores AssetKPISnapshot records for all assets. Effort: 1 day.

Task 6.2 — Executive Dashboard — Back End

Build the analytics API endpoints serving pre-computed KPI data from snapshot tables with period filtering. Effort: 1 day.

Task 6.3 — Executive Dashboard — Front End

Build Fleet Manager and Executive dashboard pages with Recharts visualizations: utilization rate bar chart, MTBF/MTTR trend lines, fuel expenditure trend line, and TCO ranked list. Effort: 2 days.

Week 7

Task 7.1 — Fuel Heatmap and Reconciliation Dashboard

Build fuel heatmap visualization and fuel reconciliation report page. Effort: 1 day.

Task 7.2 — Maintenance Cost Dashboard

Build stacked area chart for preventive vs. corrective maintenance cost trend. Effort: 0.5 days.

Task 7.3 — Unified Expiry Alert Panel

Build the alert panel with color-coded expiry items sourced from the SystemAlert table. Effort: 1 day.

Task 7.4 — Asset Service Due Panel and Incident Analytics

Build assets-due-for-service panel and incident analytics view. Effort: 1 day.

Task 7.5 — Dashboard Testing with Synthetic Data

Populate database with 12-month synthetic dataset using the seeding script and verify all KPI values render correctly. Effort: 0.5 days.

Phase 4: Weeks 8-10 — AI Model Development and Integration

Gate Condition: All three ML models meet minimum performance criteria, AI outputs visible in dashboard, replacement alerts functional.

Week 8

Task 8.1 — ML Service Setup

Initialize Python FastAPI project, configure scikit-learn and XGBoost dependencies, set up model storage directory structure. Effort: 0.5 days.

Task 8.2 — Feature Engineering — Predictive Maintenance

Write SQL queries and Python extraction scripts to build the feature dataset for the predictive maintenance model from the 12-month synthetic database. Perform exploratory data analysis. Effort: 1.5 days.

Task 8.3 — Predictive Maintenance Model Training

Train Random Forest classifier, tune hyperparameters, evaluate with cross-validation, produce model evaluation report (accuracy, precision, recall, F1-score, ROC-AUC). Serialize final model with joblib. Effort: 2 days.

Task 8.4 — Predictive Maintenance API Endpoint

Build the FastAPI `/ml/predict-maintenance` endpoint and integrate into the nightly batch job. Effort: 1 day.

Week 9

Task 9.1 — Feature Engineering — Fuel Forecasting

Build feature dataset for fuel regression model. Effort: 1 day.

Task 9.2 — Fuel Forecasting Model Training

Train XGBoost regressor, tune hyperparameters, evaluate with MAE, RMSE, and R^2. Serialize model. Effort: 1.5 days.

Task 9.3 — Feature Engineering and Training — Driver Behavior Model

Build driver behavior feature dataset, train classification model, evaluate, serialize. Effort: 1.5 days.

Task 9.4 — Replacement Alert Engine

Implement the rule-based replacement alert engine in the nightly batch job. Effort: 1 day.

Week 10

Task 10.1 — Intelligence Dashboard — Front End

Build Predictive Maintenance Scores page, Fuel Forecast page per project site, Driver Behavior Scores page with ranked driver list, and Replacement Alerts page. Effort: 2 days.

Task 10.2 — Model Evaluation Report Page

Build the model evaluation metrics display page for System Admin and Fleet Manager roles. Effort: 0.5 days.

Task 10.3 — End-to-End AI Integration Testing

Verify batch job runs completely, scores are updated nightly, dashboard reflects new scores, and high-risk alerts route correctly to Fleet Manager. Effort: 1.5 days.

Task 10.4 — System-Wide Integration Testing and Bug Fixing

Execute all TR-ST test cases, fix all Critical and High defects found. Effort: 1 day.

Phase 5: Weeks 11-12 — Documentation, UAT & Final Presentation

Gate Condition: UAT passed, final presentation delivered.

Week 11

Task 11.1 — API Documentation

Complete OpenAPI 3.0 documentation for all endpoints using swagger-jsdoc. Effort: 1 day.

Task 11.2 — User Guides

Write Fleet Manager, Site Engineer, Workshop Mechanic, and HSE Officer user guides with annotated screenshots. Effort: 1.5 days.

Task 11.3 — UAT Test Script Preparation

Write formal UAT test script document covering all Critical and High priority functional requirements with step-by-step instructions and expected outcomes. Effort: 1 day.

Task 11.4 — UAT Execution

Conduct UAT with mentor panel, record pass/fail status for each test case, log all defects found. Effort: 1.5 days.

Week 12

Task 12.1 — UAT Defect Remediation

Fix all Critical defects and as many High defects as time permits. Retest fixed items. Effort: 2 days.

Task 12.2 — Repository Cleanup and README Finalization

Ensure all code is committed, branch merged, README complete with setup instructions, .env.example documented, seed script tested from scratch. Effort: 1 day.

Task 12.3 — Final Presentation Preparation

Prepare presentation deck covering: problem statement, architecture decisions, system demo walkthrough, KPI dashboard demonstration, AI model results and evaluation metrics, and lessons learned. Effort: 1.5 days.

Task 12.4 — Final Presentation Delivery

Deliver live presentation and system demonstration to the three mentors. Effort: 0.5 days.

9.2 Summary Timeline

Phase	Weeks	Primary Focus	Gate Condition
Phase 1	1–2	Design & Architecture	SRS and ERD mentor sign-off
Phase 2	3–5	Core Web Application	All CRUD functional, RBAC enforced
Phase 3	6–7	Analytics Dashboard	Dashboard KPIs accurate and performant
Phase 4	8–10	AI Model Integration	ML models meet minimum F1/R^2 criteria
Phase 5	11–12	UAT and Presentation	UAT passed, final presentation delivered

---

10. Appendix: UI Wireframe Blueprints

The responsive web panels, mobile-optimized driver screens, and telemetry dashboard maps can be reviewed via our interactive asset canvas:

Project UI Wireframe file ↗